
MENU-DRIVEN STRING OPERATIONS PROGRAM

Submitted By

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a menu-driven C program to perform the following operations on a string, with each operation implemented using a separate function:

1. Count the characters in a string without using any built-in length function.
2. Print each character of a string on a new line.
3. Count the vowels and consonants in a string.
4. Exit the program.

2. Objectives

- To understand how strings are stored in C as character arrays terminated by the null character (`\0`).
- To practise traversing a string using loops and accessing its characters by index.
- To write modular programs using user-defined functions with parameters and return values.
- To create a menu-driven program using a do-while loop and a switch statement.
- To perform basic input validation so the program does not fail on wrong input.

3. Concepts Used

- **Strings and character arrays:** the input is stored in a char array of size 200.
- **Null character:** every C string ends with `\0`; this is used to detect the end of the string, and so to find its length.
- **Loops (while, for, do-while):** used for traversing the string and for repeating the menu.
- **Decision making (if-else, switch):** used to classify characters and to select the menu option.
- **Functions:** each operation is written as a separate function, which keeps the program neat and reusable.
- **ASCII values:** uppercase letters are converted to lowercase by adding 32, so that vowels are checked in a single form.

4. Functions Used

Function	Return Type	Purpose
<code>countCharacters()</code>	int	Counts characters by traversing the string until the null character is found (no built-in length function used).
<code>printCharacters()</code>	void	Prints every character of the string on a separate line.
<code>countVowelsConsonants()</code>	void	Counts and displays the number of vowels and consonants in the string.
<code>readString()</code>	void	Reads a full line (including spaces) from the user and removes the trailing newline.
<code>displayMenu()</code>	void	Displays the menu of available operations.
<code>main()</code>	int	Runs the menu loop and calls the appropriate function for the chosen option.

5. Algorithm

5.1 Main Program

1. Start.
2. Display the menu and read the choice.
3. If the choice is 1, 2 or 3, read a string and call the corresponding function.
4. If the choice is 4, display the exit message and stop.
5. For any other choice, display an "Invalid choice" message.
6. Repeat from step 2 until the choice is 4.

5.2 Count Characters

1. Set `count = 0`.
2. While `str[count]` is not `'\0'`, increase `count` by 1.
3. Return `count`.

5.3 Print Each Character

1. If the string is empty, display a message and return.
2. For `i = 0` until `str[i]` is `'\0'`, print `str[i]` followed by a new line (a space is shown as `[space]`).

5.4 Count Vowels and Consonants

1. Set `vowels = 0` and `consonants = 0`.
2. For each character, convert it to lowercase if it is an uppercase letter.
3. If the character is between `'a'` and `'z'`: if it is `a`, `e`, `i`, `o` or `u` increase `vowels`, otherwise increase `consonants`.

4. Ignore digits, spaces and special symbols.
5. Display the vowel and consonant counts.

6. Flowchart

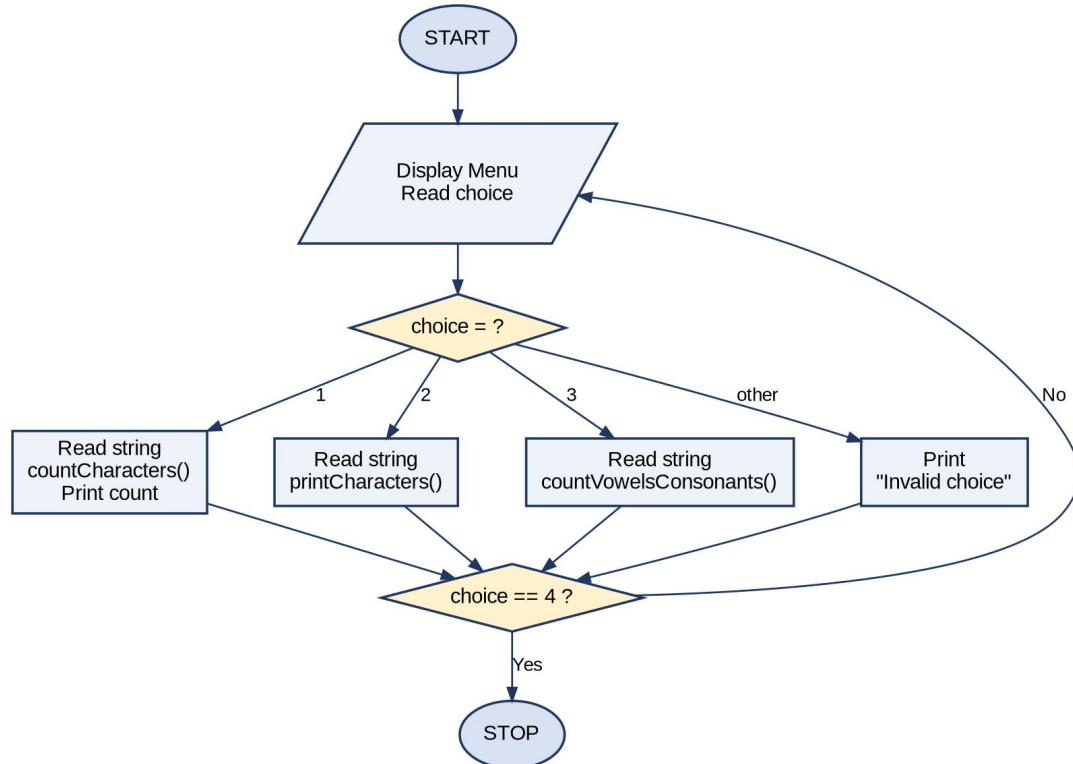


Figure 1: Flowchart of the menu-driven program

7. Source Code

File name: string_operations.c (compiled with GCC using gcc -Wall -Wextra with no warnings).

```
/*
 * Program : Menu-Driven String Operations
 * Author  : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 *
 * Description:
 * A menu-driven program that performs the following operations
 * on a string, each implemented in a separate function:
 * 1. Count characters in a string (without built-in length functions)
 * 2. Print each character of a string on a new line
 * 3. Count vowels and consonants in a string
 * 4. Exit
 */

#include <stdio.h>

#define MAX 200 /* maximum size of the input string buffer */

/* Function prototypes */
int countCharacters(const char str[]);
```

```

void printCharacters(const char str[]);
void countVowelsConsonants(const char str[]);
void readString(char str[]);
void displayMenu(void);

/* -----
 * Function : countCharacters
 * Purpose  : Counts the characters of a string manually by traversing
 *            it until the null character '\0' is found.
 * Returns   : Number of characters in the string.
 * ----- */
int countCharacters(const char str[])
{
    int count = 0;

    while (str[count] != '\0')
    {
        count++;
    }
    return count;
}

/* -----
 * Function : printCharacters
 * Purpose  : Prints every character of the string on a new line.
 * ----- */
void printCharacters(const char str[])
{
    int i;

    if (str[0] == '\0')
    {
        printf("The string is empty. Nothing to print.\n");
        return;
    }

    printf("Characters of the string:\n");
    for (i = 0; str[i] != '\0'; i++)
    {
        if (str[i] == ' ')
            printf("[space]\n");      /* make spaces visible */
        else
            printf("%c\n", str[i]);
    }
}

/* -----
 * Function : countVowelsConsonants
 * Purpose  : Counts vowels and consonants in the string.
 *            Only alphabets are considered; digits, spaces and
 *            special symbols are ignored.
 * ----- */
void countVowelsConsonants(const char str[])
{
    int i;
    int vowels = 0, consonants = 0;
    char ch;

    for (i = 0; str[i] != '\0'; i++)
    {
        ch = str[i];

        /* convert uppercase letter to lowercase */
        if (ch >= 'A' && ch <= 'Z')
            ch = ch + 32;

        if (ch >= 'a' && ch <= 'z')
        {

```

```

        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
            vowels++;
        else
            consonants++;
    }
}

printf("Number of vowels      : %d\n", vowels);
printf("Number of consonants : %d\n", consonants);
}

/* -----
 * Function : readString
 * Purpose  : Reads a line (including spaces) from the user and
 *            removes the trailing newline character.
 * ----- */
void readString(char str[])
{
    int i;

    printf("Enter a string: ");
    if (fgets(str, MAX, stdin) == NULL)
    {
        str[0] = '\0';
        return;
    }

    /* remove the newline left by fgets */
    for (i = 0; str[i] != '\0'; i++)
    {
        if (str[i] == '\n')
        {
            str[i] = '\0';
            break;
        }
    }
}

/* -----
 * Function : displayMenu
 * Purpose   : Displays the menu of operations.
 * ----- */
void displayMenu(void)
{
    printf("\n===== STRING OPERATIONS MENU =====\n");
    printf("1. Count characters in a string\n");
    printf("2. Print each character on a new line\n");
    printf("3. Count vowels and consonants\n");
    printf("4. Exit\n");
    printf("===== \n");
    printf("Enter your choice (1-4): ");
}

/* -----
 * Function : main
 * Purpose   : Drives the program using a do-while loop and switch.
 * ----- */
int main(void)
{
    char str[MAX];
    int choice;
    int result;

    do
    {
        displayMenu();

        if (scanf("%d", &choice) != 1)
        {

```

```

        /* invalid (non-numeric) input: clear buffer and retry */
        while (getchar() != '\n')
            ;
        printf("Invalid input! Please enter a number between 1 and 4.\n");
        choice = 0;
        continue;
    }
    while (getchar() != '\n') /* discard leftover characters */
        ;

    switch (choice)
    {
        case 1:
            readString(str);
            result = countCharacters(str);
            printf("Total number of characters: %d\n", result);
            break;

        case 2:
            readString(str);
            printCharacters(str);
            break;

        case 3:
            readString(str);
            countVowelsConsonants(str);
            break;

        case 4:
            printf("Exiting the program. Thank you!\n");
            break;

        default:
            printf("Invalid choice! Please enter a number between 1 and 4.\n");
    }
    } while (choice != 4);

    return 0;
}

```

8. Sample Output

The program was compiled and run, and the following outputs were obtained. The text after each prompt is the input typed by the user.

Menu displayed by the program

```

===== STRING OPERATIONS MENU =====
1. Count characters in a string
2. Print each character on a new line
3. Count vowels and consonants
4. Exit
=====
Enter your choice (1-4):

```

Operation 1: Count characters

```

Enter your choice (1-4): 1
Enter a string: Hello World
Total number of characters: 11

```

Operation 2: Print each character on a new line

```
Enter your choice (1-4): 2
Enter a string: C Lang
Characters of the string:
C
[space]
L
a
n
g
```

Operation 3: Count vowels and consonants

```
Enter your choice (1-4): 3
Enter a string: Programming in C 2026
Number of vowels      : 4
Number of consonants : 10
```

Invalid choice and invalid input

```
Enter your choice (1-4): 9
Invalid choice! Please enter a number between 1 and 4.

Enter your choice (1-4): abc
Invalid input! Please enter a number between 1 and 4.
```

Operation 4: Exit

```
Enter your choice (1-4): 4
Exiting the program. Thank you!
```

9. Test Cases

#	Option	Input	Expected Output	Result
1	1	Hello World	Total number of characters: 11	Pass
2	1	(empty string)	Total number of characters: 0	Pass
3	1	C is fun!	Total number of characters: 9	Pass
4	2	C Lang	C, [space], L, a, n, g (each on a new line)	Pass
5	3	Programming in C 2026	Vowels: 4, Consonants: 10	Pass
6	3	AEIOU aeiou	Vowels: 10, Consonants: 0	Pass
7	3	rhythm 123 !@#	Vowels: 0, Consonants: 6	Pass
8	9	(invalid number)	Invalid choice message; menu shown again	Pass
9	abc	(non-numeric)	Invalid input message; menu shown again	Pass

#	Option	Input	Expected Output	Result
10	4	-	Exit message; program terminates	Pass

10. Conclusion

The program successfully performs all the required string operations through a simple menu. The length of the string is found manually by searching for the null character, without any built-in length function. Each operation is placed in its own function, which makes the code modular and easy to read, test and modify. Input validation ensures that invalid menu choices or non-numeric input do not stop the program. Through this project, the concepts of strings, loops, functions and menu-driven programming in C were understood and applied.